arXiv:2304.14997v4 [cs.LG] 28 Oct 2023

Towards Automated Circuit Discovery for Mechanistic Interpretability

Arthur Conmy[*] Independent
Augustine N. Mavor-Parker[*] UCL
Aengus Lynch[*] UCL
Stefan Heimersheim University of Cambridge
Adrià Garriga-Alonso[*] FAR AI

Abstract

Through considerable effort and intuition, several recent works have reverse-engineered nontrivial behaviors of transformer models. This paper systematizes the mechanistic interpretability process they followed. First, researchers choose a metric and dataset that elicit the desired model behavior. Then, they apply activation patching to find which abstract neural network units are involved in the behavior. By varying the dataset, metric, and units under investigation, researchers can understand the functionality of each component.

We automate one of the process' steps: finding the connections between the abstract neural network units that form a circuit. We propose several algorithms and reproduce previous interpretability results to validate them. For example, the ACDC algorithm rediscovered 5/5 of the component types in a circuit in GPT-2 Small that computes the Greater-Than operation. ACDC selected 68 of the 32,000 edges in GPT-2 Small, all of which were manually found by previous work. Our code is available at https://github.com/ArthurConmy/Automatic-Circuit-Discovery.

1 Introduction

Rapid progress in transformer language modelling (Vaswani et al., 2017; Devlin et al., 2019; OpenAI, 2023, *inter alia*) has directed attention towards understanding the causes of new capabilities (Wei et al., 2022) in these models. Researchers have identified precise high-level predictors of model performance (Kaplan et al., 2020), but transformers are still widely considered 'black-boxes' (Alishahi, Chrupała, and Linzen, 2019) like almost all other neural network models (Fong and Vedaldi, 2017; Buhrmester, Münch, and Arens, 2021).[2] Interpretability research aims to demystify machine learning models, for example by explaining model outputs in terms of domain-relevant concepts (Zhang et al., 2021).

Mechanistic interpretability focuses on reverse-engineering model components into human-understandable algorithms (Olah, 2022). Much research in mechanistic interpretability views models as a computational graph (Geiger et al., 2021), and circuits are subgraphs with distinct functionality (Wang et al., 2023). The current approach to extracting circuits from neural networks relies on a lot of manual inspection by humans (Räuker et al., 2022). This is a major obstacle to scaling up mechanistic interpretability to larger models, more behaviors, and complicated behaviors composed of many sub-circuits. This work identifies a workflow for circuit research, and automates part of it by presenting several methods to extract computational graphs from neural networks.

Our main contributions are as follows. First, we systematize the common workflow prevalent in many existing mechanistic interpretability works, outlining the essential components of this process

[*]Work partially done at Redwood Research. Correspondence to arthurconmy@gmail.com
[2]Though this perspective is not universal (Lipton, 2016).

37th Conference on Neural Information Processing Systems (NeurIPS 2023).

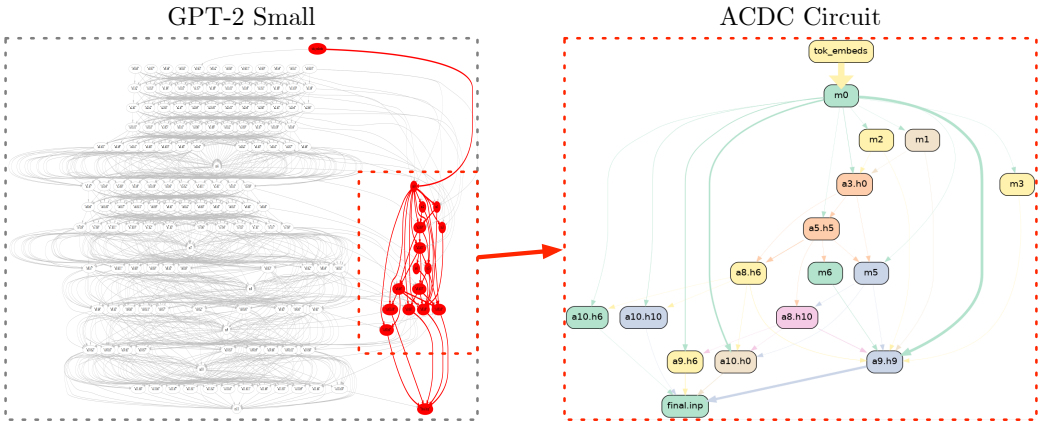

Figure 1: **Automatically discovering circuits with ACDC.** *Left:* a computational graph for GPT-2 Small, with a recovered circuit for the IOI task highlighted in red. Only edges between adjacent layers are shown. *Right:* the recovered circuit with labelled nodes. All heads recovered were identified as part of the IOI circuit by Wang et al. (2023). Edge thickness is proportional to importance.

(Section 2). One of its steps is to find a subgraph of the model which implements the behavior of interest, which is a step possible to automate. We introduce Automatic Circuit DisCovery (ACDC), a novel algorithm that follows the way in which researchers identify circuits (Section 3), and adapt Subnetwork Probing (SP; Cao, Sanh, and Rush, 2021) and Head Importance Score for Pruning (HISP; Michel, Levy, and Neubig, 2019) for the same task. Finally, we introduce quantitative metrics to evaluate the success of circuit extraction algorithms (Sections 4 and 4.2). We present a detailed ablation study of design choices in Appendix E and qualitative studies in Appendices F, G, H, I and J.

2 The Mechanistic Interpretability Workflow

Mechanistic interpretability attempts to explain and predict neural network behaviors by understanding the underlying algorithms implemented by models. In the related work section we discuss the mechanistic interpretability field and its relationship to 'circuits' research (Section 5). Neural network behaviors are implemented by algorithms within the model's computational graph, and prior work has identified subgraphs (*circuits*, following Wang et al. (2023)'s definition) that capture the majority of particular behaviors. In this section, we describe a workflow that several prior works have followed that has been fruitful for finding circuits in models.

As a concrete example of an approach taken to finding a circuit, Hanna, Liu, and Variengien (2023) prompt GPT-2 Small with a dataset of sentences like "The war lasted from 1517 to 15". GPT-2 Small completes this sentence with "18" or "19" or any larger two digit number, but not with any two digit number that is at most "17" (from here, we refer to prompt completions like this as the "Greater-Than" task). This behavior can be measured by the difference in probability the model places on a completion "18" or "19" or larger and the probability the model places on a completion "17" or smaller. Note that we use the term 'dataset' to refer to a collection of prompts that elicit some behavior in a model: we do not train models on these examples, as in this paper we focus on post-hoc interpretability.

The researchers then create a *corrupted dataset* of sentences that do not have any bias against particular two digit completions (the '01-dataset' (Hanna, Liu, and Variengien, 2023)). The researchers attribute the greater-than operation to late layer MLPs and then find earlier components that identify the numerical values of years, including attention heads in the model. Finally, Hanna, Liu, and Variengien (2023) interpret the role of each set of components. For example, they identify early model components that respond to the "17" token, and later model components that boost the importance of logits for years greater than 17.

There are equivalent steps taken in a growing number of additional works (Heimersheim and Janiak, 2023, the "Docstring" task; Goldowsky-Dill et al., 2023, the "Induction" task; Wang et al., 2023,

the "IOI" task), described in brief in Table 1 and in detail in Appendices F, H and J. We identify the workflow that eventually finds a circuit as following three steps. Researchers:

1. Observe a behavior (or task[3]) that a neural network displays, create a dataset that reproduces the behavior in question, and choose a metric to measure the extent to which the model performs the task.
2. Define the scope of the interpretation, i.e. decide to what level of granularity (e.g. attention heads and MLP layers, individual neurons, whether these are split by token position) at which one wants to analyze the network. This results in a computational graph of interconnected model units.
3. Perform an extensive and iterative series of patching experiments with the goal of removing as many unnecessary components and connections from the model as possible.

Researchers repeat the previous three steps with a slightly different dataset or granularity, until they are satisfied with the explanation of the circuit components.

This work (ACDC) presents a tool to fully automate Step 3. Before we dive into the details of ACDC, we expand on what Steps 1-3 involve, and review examples from previous work that we use to evaluate ACDC.

2.1 Step 1: Select a behavior, dataset, and metric

The first step of the general mechanistic interpretability workflow is to choose a neural network behavior to analyze. Most commonly researchers choose a clearly defined behavior to isolate only the algorithm for one particular task, and curate a dataset which elicits the behavior from the model. Choosing a clearly defined behavior means that the circuit will be easier to interpret than a mix of circuits corresponding to a vague behavior. Some prior work has reverse-engineered the algorithm behind a small model's behavior on all inputs in its training distribution (Nanda et al., 2023; Chughtai, Chan, and Nanda, 2023), though for language models this is currently intractable, hence the focus on individual tasks.

We identified a list of interesting behaviors that we used to test our method, summarized in Table 1. These include previously analyzed transformer models (1 and 3 on GPT-2 Small, 2 and 6 on smaller language transformers) where researchers followed a workflow similar to the one we described above. Tasks 4 and 5 involve the full behavior of tiny transformers that implement a known algorithm, compiled with `tracr` (Lindner et al., 2023). For each task, we mention the metric used in previous work to measure the extent to which the model performs the task on the corresponding dataset.

2.2 Step 2: Divide the neural network into a graph of smaller units

To find circuits for the behavior of interest, one must represent the internals of the model as a computational directed acyclic graph (DAG, e.g. Figure 2a). Current work chooses the abstraction level of the computational graph depending on the level of detail of their explanations of model behavior. For example, at a coarse level, computational graphs can represent interactions between attention heads and MLPs. At a more granular level they could include separate query, key and value activations, the interactions between individual neurons (see Appendix I), or have a node for each token position (Wang et al., 2023).

Node connectivity has to be faithful to the model's computation, but that does not fully specify its definition. For example, following Elhage et al. (2021), many works consider the connections between model components in non-adjacent layers due to the additivity of the residual stream, even though these are computed with dynamic programming in the actual model implementation. Connectivity defines what is considered a direct or a mediated interaction (Pearl, 2009; Vig et al., 2020). See for example Figure 2a, where component B has both a direct effect on the output node O and an indirect effect on the output through component A.

[3]Section 3 formally defines "task". We use "behavior" and "task" interchangeably.